

第03章：LangSmith基本使用

讲师：尚硅谷-宋红康

官网：[尚硅谷](#)

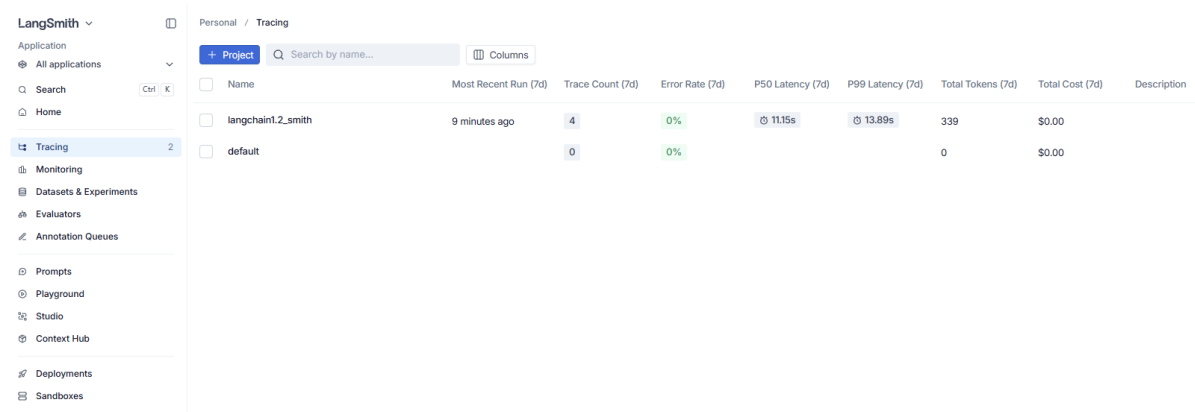
1、LangSmith概述

1.1 什么是LangSmith?

LangSmith 是 LangChain 生态系统中专门用于 LLM（大语言模型）应用**调试、监控、评估和管理**的平台。

- 🔍 **追踪(tracing)**: 记录每次 LLM 调用的详细信息
- 📊 **监控(monitoring)**: 实时查看应用性能
- 🪲 **调试(debug)**: 排查问题和优化性能
- 📈 **评估(evaluate)**: 系统化测试 LLM 应用

主界面：



Name	Most Recent Run (7d)	Trace Count (7d)	Error Rate (7d)	P50 Latency (7d)	P99 Latency (7d)	Total Tokens (7d)	Total Cost (7d)	Description
langchain1.2_smith	9 minutes ago	4	0%	11.15s	13.89s	339	\$0.00	
default		0	0%			0	\$0.00	

1.2 具体功能

功能1：核心应用与开发

1、Tracing（追踪）

- 功能：这是 LangSmith 最核心的功能。它会完整记录你大模型应用的每一次调用链路（Trace）。
- 作用：当你的 Agent（智能体）或 RAG 系统运行变慢或报错时，点击进入对应的项目（如上图中的 `langchain1.2_smith`），你可以看到每一步具体的 Prompt 是什么、模型返回了什么、消耗了多少 Token，以及每一个链条节点的耗时，非常方便**排查 Bug 和优化性能**。

2、Monitoring（监控）

- 功能：提供生产环境的高级数据可视化看板。
- 作用：帮你从宏观角度监控应用在一段时间内的运行状况。你可以看到 **Token 消耗趋势、QPS（每秒请求数）、错误率、平均延迟（Latency）以及成本预估**。适合应用上线后观察系统的稳定性和开销。

3、Datasets & Experiments（数据集与实验）

- 功能：用于管理测试数据集并运行对比实验。

- 作用：你可以把用户的真实输入、特定的边界情况（Edge Cases）存为数据集。当你修改了 Prompt 或更换了底层大模型时，可以在这里运行**自动化对比测试**，直观看到新旧版本在同一批测试集上的表现差异。

4、Evaluators（评估器）

- 功能：配置和自动化评估任务。
- 作用：大模型的输出往往难以用传统的断言（Assert）来测试。这里允许你配置基于规则（如关键词匹配）或基于模型（LLM-as-a-judge）的评估指标（如：答案相关性、是否包含幻觉等），对追踪到的数据或实验结果进行自动打分。

5、Annotation Queues（标注队列）

- 功能：人工反馈与数据清洗工具。
- 作用：在应用开发或初上线阶段，你可以把一部分痕迹（Traces）发送到标注队列中，让团队中的核心成员、业务专家或人工客服进行**手动打分、纠正回答或贴标签**，这些高质量的人工标注数据后续可直接用于微调模型或充当测试集。

功能2：提示词与调试工具

1、Prompts（提示词管理）

- 功能：类似“提示词版的 GitHub”。
- 作用：把 Prompt 从代码中解耦出来，统一在云端管理。你可以在这里对 Prompt 进行版本控制（如 `v1`、`v2`），直接在代码中通过 API 动态拉取最新的提示词。它还支持团队协作和 Prompt 的分享。

2、Playground（演练场）

- 功能：一个网页端的模型交互界面。
- 作用：无需写任何代码，直接在这里选择不同的模型（如 OpenAI、Anthropic 或是本地模型），快速微调并测试你的 Prompt 效果，还可以一键将调整好的 Prompt 保存到上方的 Prompts 仓库中。

3、Studio（工作室）

- 功能：通常与 LangGraph 深度集成，提供可视化的图形交互界面。
- 作用：如果你的应用是基于图结构（Graph-based）的复杂复杂 Agent 架构，Studio 可以让你**可视化地看到状态机（State）在各个节点之间的流转**，甚至支持在某个节点“暂停”，手动修改数据后再继续向下执行，是调试复杂智能体交互的利器。

4、Context Hub（上下文中心）

- 功能：管理全局上下文或通用组件配置。
- 作用：用于存放可在多个项目或 Prompt 中复用的公共上下文模板、全局变量或系统预设提示。

功能3：部署与沙盒

1、Deployments（部署）

- 功能：一键将你的 LangChain 应用或 LangGraph Agent 部署为线上可用的 API 服务（通常依托于 LangGraph Cloud）。
- 作用：提供开箱即用的生产端点，帮你处理高并发、队列管理和状态持久化，让你专注于编写业务逻辑。

2、Sandboxes（沙盒）

- 功能：提供轻量级的在线运行和测试环境。
- 作用：在不污染生产环境的前提下，供开发人员安全地试运行、测试新部署的 Agent 或执行自动化脚本。

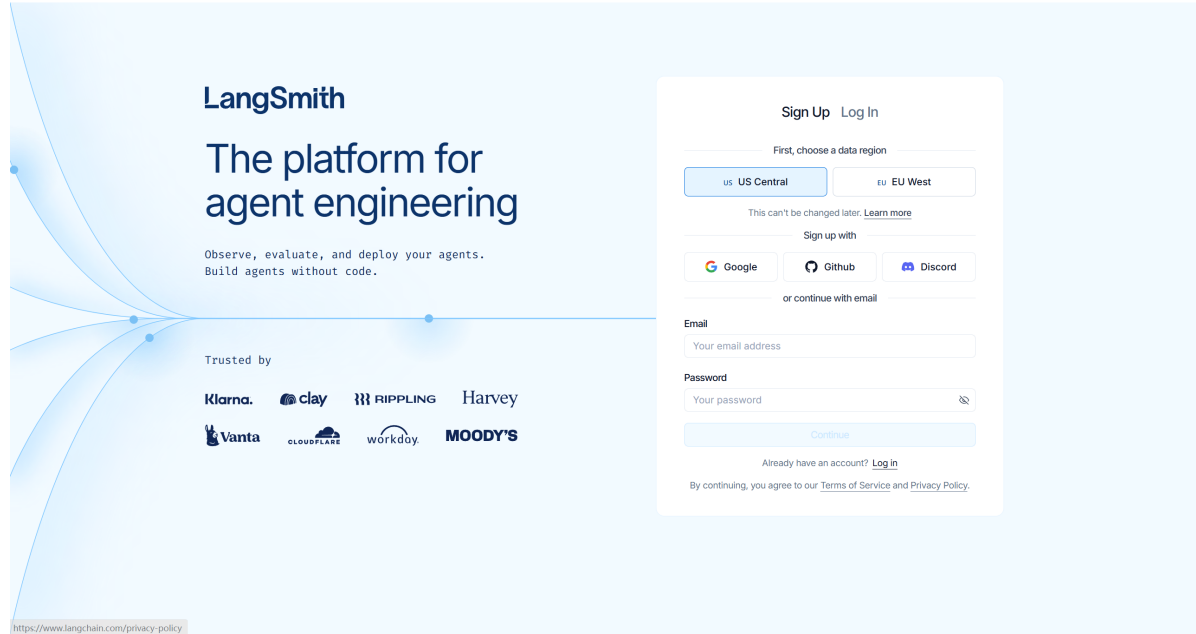
建议：现阶段大家可以重点关注 **Tracing**（观察你的项目里的调用细节）和 **Playground**（快速调优提示词）。当你的应用结构开始走向复杂（比如引入了复杂的 RAG 检索或多 Agent 协同）时，再逐步引入 **Datasets** 进行量化评估，并利用 **Studio** 进行可视化调试。

2、准备账号

2.1 注册或登录

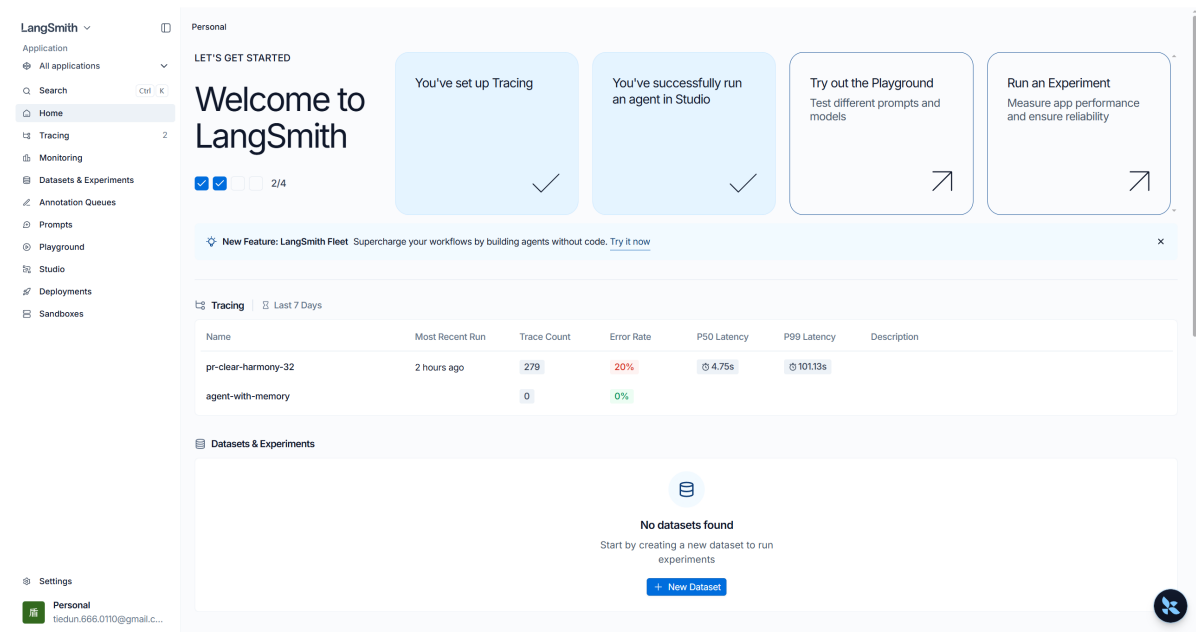
步骤1：访问官网

访问Langsmith官网：<https://smith.LangChain.com/>



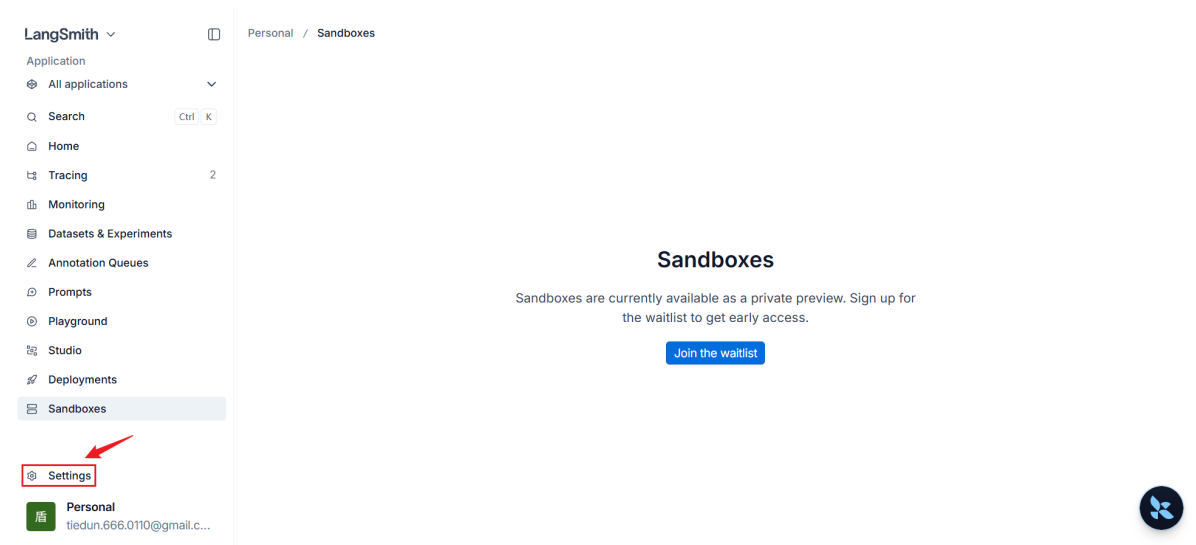
步骤2：自由选择注册或登录方式

步骤3：登录成功

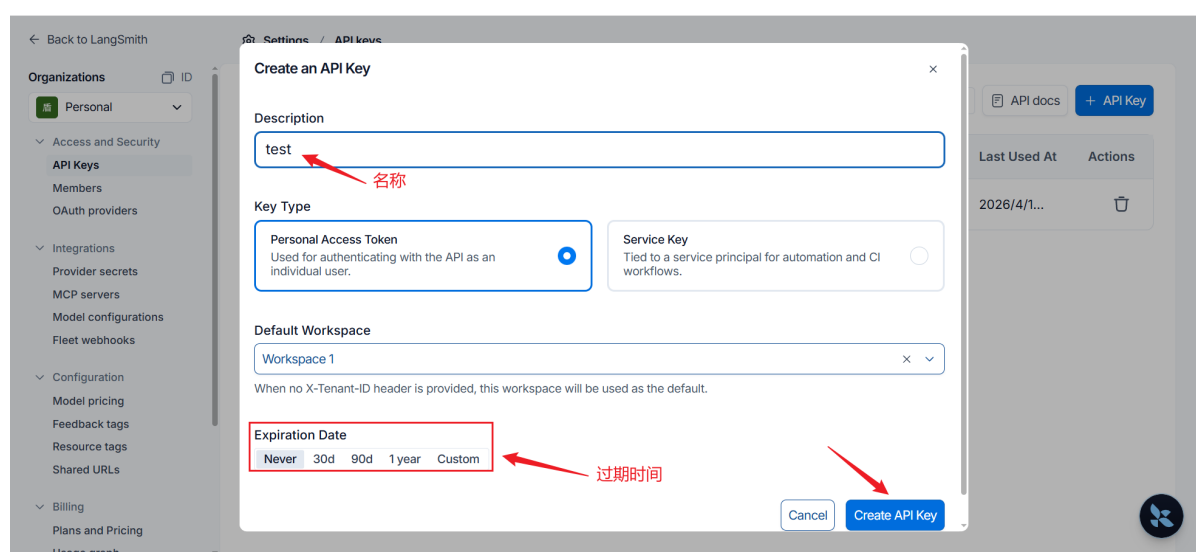
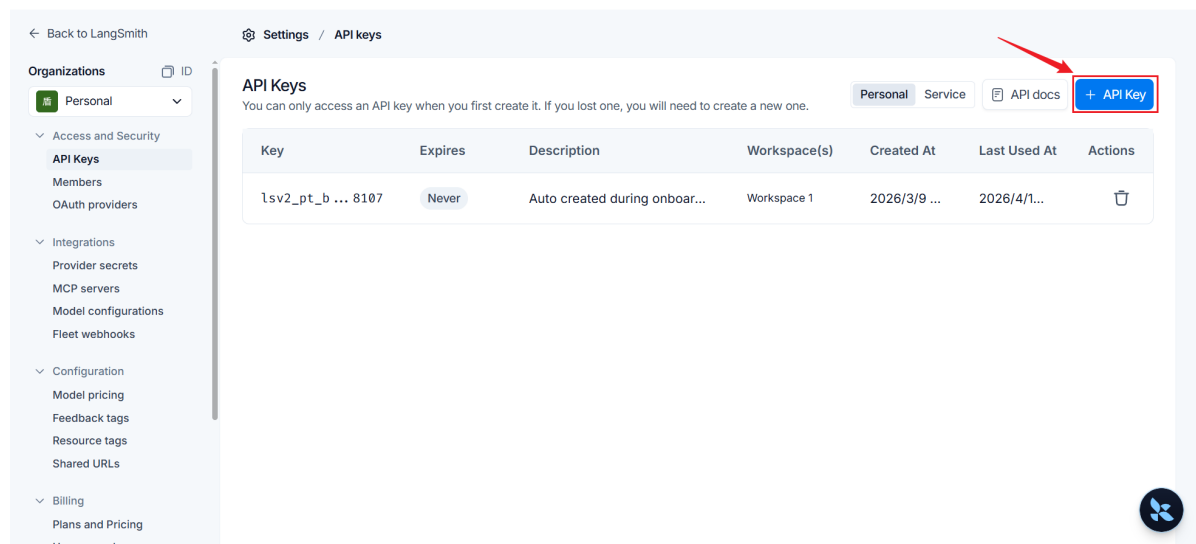


2.2 获取API_KEY

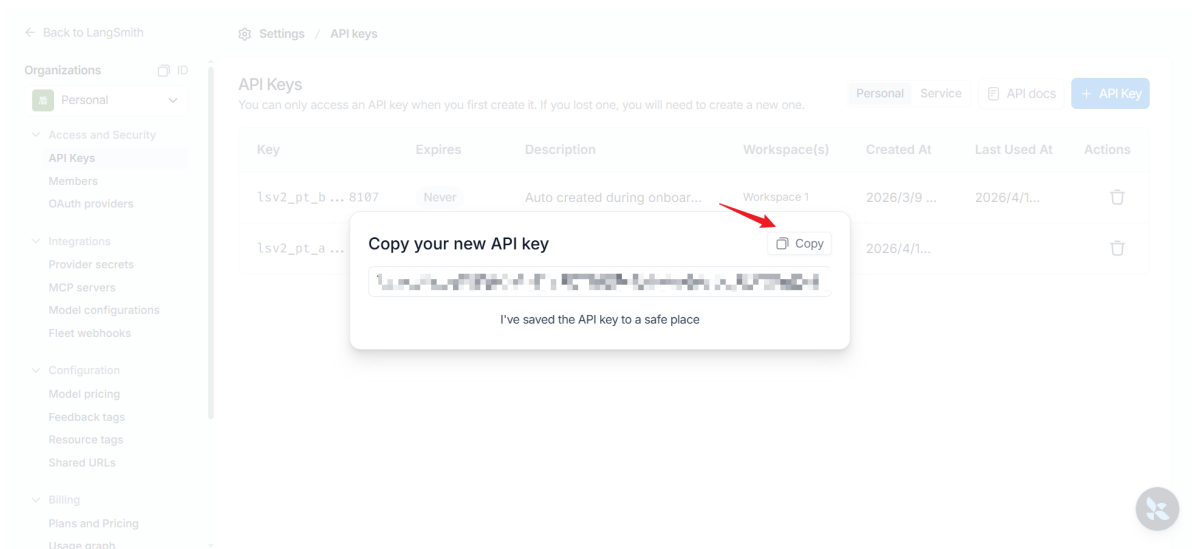
步骤1: 打开设置



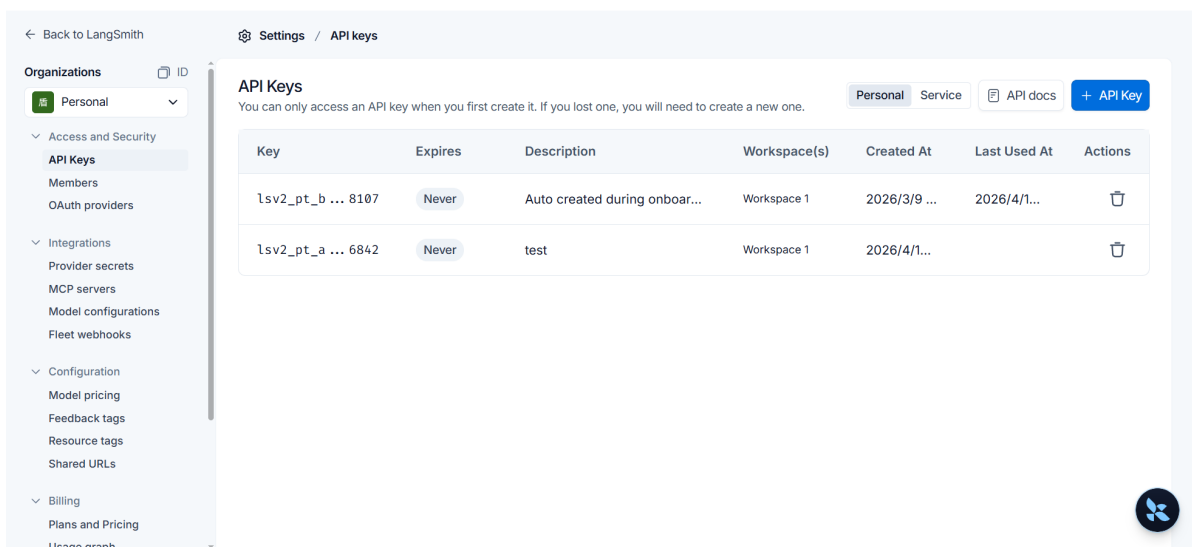
步骤2: 创建API_KEY



步骤3: 保存KEY

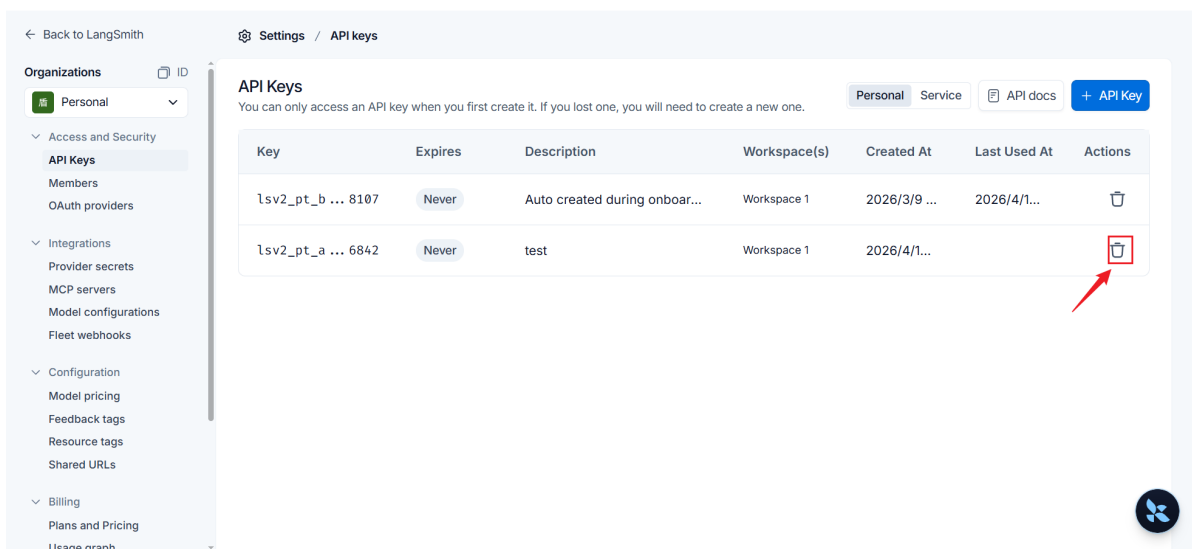


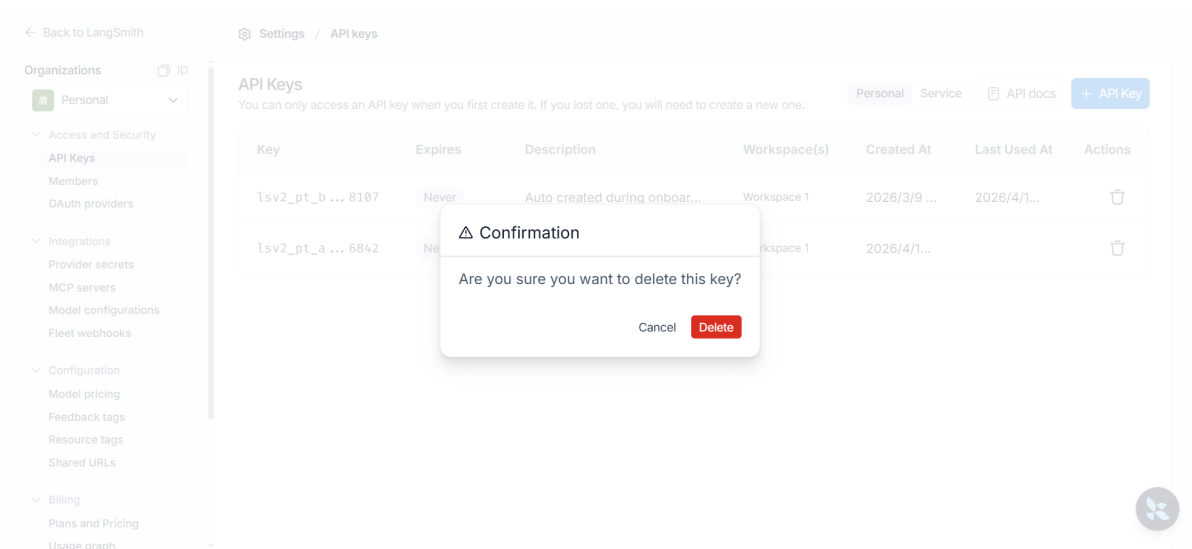
注意：点击 **copy** 将API_KEY保存到剪贴板并关闭弹窗，API_KEY只在上述窗口出现一次，关闭弹窗后回到设置页面，此后就无法在官网查看API_KEY的内容了，请妥善保存。



步骤4（可选）：按需删除

如果需要删除KEY，点击右侧图标即可。





2.3 新增环境变量

在.env配置文件中，添加四个环境变量：

```
1 # 是否启用Langsmith监控功能
2 LANGSMITH_TRACING=true
3
4 # Langsmith监控webUI地址
5 LANGSMITH_ENDPOINT=https://api.smith.LangChain.com
6
7 # 创建的API_KEY
8 LANGSMITH_API_KEY=<YOUR_API_KEY>
9
10 # 自定义项目名称，可以在Langsmith webUI监控页面根据名称查看对应的运行记录
11 LANGSMITH_PROJECT="pr-clear-harmony-32"
```

3、查看监控指标

添加上述环境变量后，在程序中通过load_dotenv()加载，而后运行LangChain代码，LangSmith会自动记录运行指标，并同步至后台服务，我们可以在LangSmith官网查看运行记录。

步骤1：运行任意LangChain程序

举例1：

```
1 import os
2
3 from dotenv import load_dotenv
4 from langchain_deepseek import ChatDeepSeek
5
6 # 将env文件中的变量加载为环境变量
7 #override=True: 表示.env优先
8 load_dotenv(override=True)
9
10 DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY")
11 DEEPSEEK_BASE_URL = os.getenv("DEEPSEEK_BASE_URL")
12
13 model = ChatDeepSeek(
14     api_key=DEEPSEEK_API_KEY,
15     api_base=DEEPSEEK_BASE_URL,
16     model_name="deepseek-v4-flash"
```

```
17 )
18
19 print(model.invoke("你好"))
```

举例2:

```
1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4
5 load_dotenv(override=True)
6
7 CLOSEAI_API_KEY=os.getenv("CLOSEAI_API_KEY")
8 CLOSEAI_BASE_URL=os.getenv("CLOSEAI_BASE_URL")
9
10 model = init_chat_model(model="deepseek-v4-flash",
11                          model_provider="openai",
12                          api_key=CLOSEAI_API_KEY,
13                          base_url=CLOSEAI_BASE_URL)
14
15 print(model.invoke("你好，用一句话回答"))
```

举例3:

```
1 from langchain.chat_models import init_chat_model
2 from dotenv import load_dotenv
3 import os
4 from rich import print as rprint
5
6 # 从.env文件中加载环境变量
7 load_dotenv(override=True)
8
9 DEEPSEEK_API_KEY = os.getenv("DEEPSEEK_API_KEY")
10 DEEPSEEK_BASE_URL = os.getenv("DEEPSEEK_BASE_URL")
11
12 # 1. 初始化模型
13 model = init_chat_model(
14     model="deepseek-v4-flash",
15     model_provider="deepseek",
16     api_key=DEEPSEEK_API_KEY,
17     base_url=DEEPSEEK_BASE_URL,
18     temperature=0.2,
19     max_tokens=500,
20     # 指定可调整参数
21     configurable_fields=("model", "model_provider", "temperature",
22                          "max_tokens"),
23 )
24
25 # 2. 准备 config 字典
26 config = {
27     "run_name": "joke_generation", # 在LangSmith中这次运行会显示为
28     "joke_generation"
29     "tags": ["my_tag1", "my_tag2"], # 打上标签便于分类查找
30     "metadata": {
31         "user_id": "shkstart", # 记录用户ID
32         "session_id": "sess_123" # 记录会话ID
33     }
34 }
```

```

31     },
32     "configurable": {
33         "model": "deepseek-v4-pro", # 配置模型参数
34         "model_provider": "openai", # 配置模型提供商参数
35         "temperature": 0.7, # 配置温度参数
36         "max_tokens": 1000 # 配置最大令牌数
37     }
38 }
39
40 # 3. 调用模型并传入config
41 response = model.invoke(
42     "1 + 2 = ? ",
43     config=config
44 )
45
46 rprint(response)

```

步骤2：打开监控界面

LangSmith Personal / Tracing

Application: All applications

Search: Ctrl K

Home

Tracing 2

Monitoring

Datasets & Experiments

Annotation Queues

Prompts

Playground

Studio

Deployments

Sandboxes

Settings

Personal tiedun.666.0110@gmail.c...

Name	Most Recent Run (7d)	Trace Count (7d)	Error Rate (7d)	P50 Latency (7d)	P99 Latency (7d)	Total Traces
pr-clear-harmony-32	3 hours ago	279	20%	4.75s	101.13s	131,221
agent-with-memory		0	0%			0

此时在LangSmith官方WebUI的Tracing界面下，可以看到按照 `LANGSMITH_PROJECT` 命名的项目。

步骤3：查看运行指标

点击条目任意位置可以进入详情页面

LangSmith Personal / Tracing / pr-clear-harmony-32

pr-clear-harmony-32 ID

Retention 14d Dashboard + New

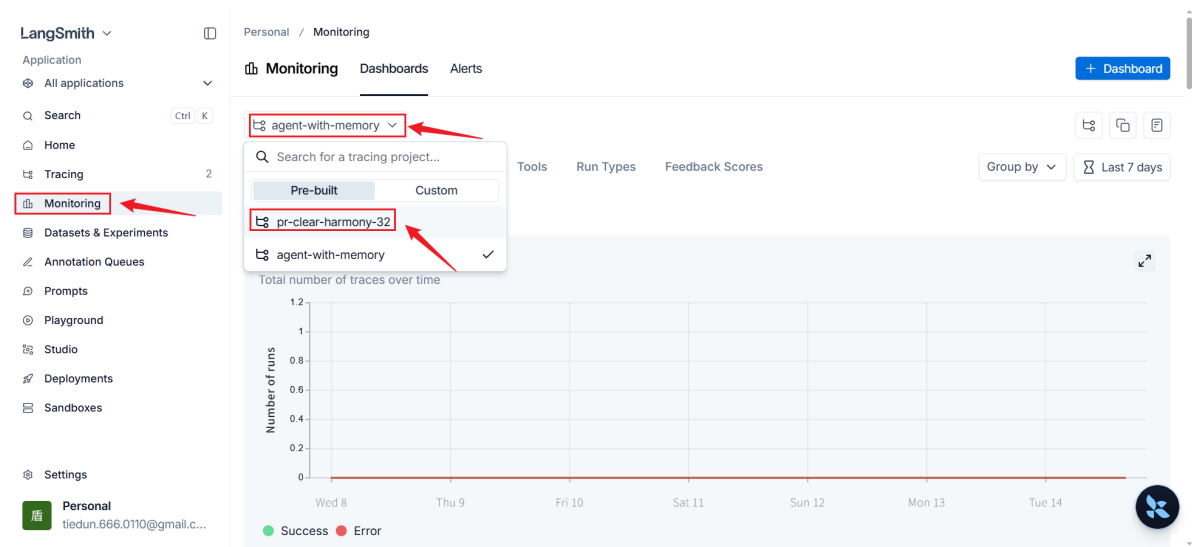
Runs Threads Evaluators Automations Insights

Default View Last 1 day Traces Runs Add filter

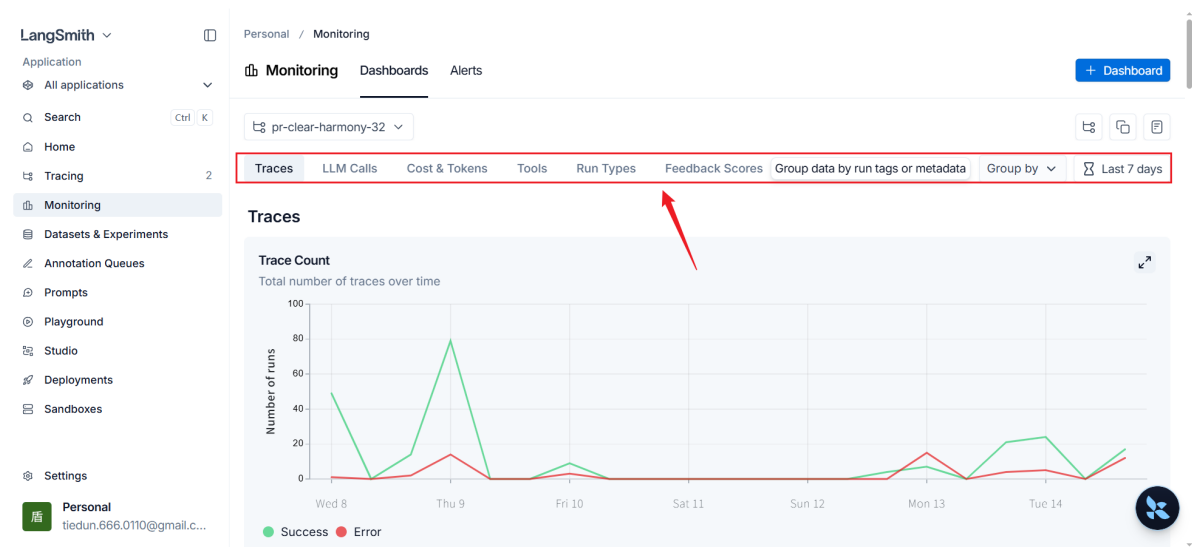
Name	Input	Output	Error	Start Time	Latency	Dataset
ChatOpenAI	human: 花儿为什么这...	ai: 这是个有趣的问...		4/15/2026, 2...	17.86s	
ChatOpenAI	human: 花儿为什么这...	ai: 这是一个很有趣...		4/15/2026, 11...	19.22s	
ChatOpenAI	human: 花儿为什么这...	ai: "花儿为什么这...		4/15/2026, 11...	14.40s	
ChatDeepSeek	human: 花儿为什么这...	ai: "花儿为什么这...		4/15/2026, 11...	35.40s	
ChatDeepSeek	human: 花儿为什么这...	ai: "花儿为什么这...		4/15/2026, 11...	36.71s	
ChatOpenRouter	human: 这张图里有什...	ai: 这张图显示了一...		4/15/2026, 11...	2.25s	
ChatOpenRouter	human: 这张图里有什...	ai: 这张图片中有一...		4/15/2026, 11...	3.24s	

此处列出了详细的运行指标，点击某次运行记录，可以查看更详细的信息，自行探索。

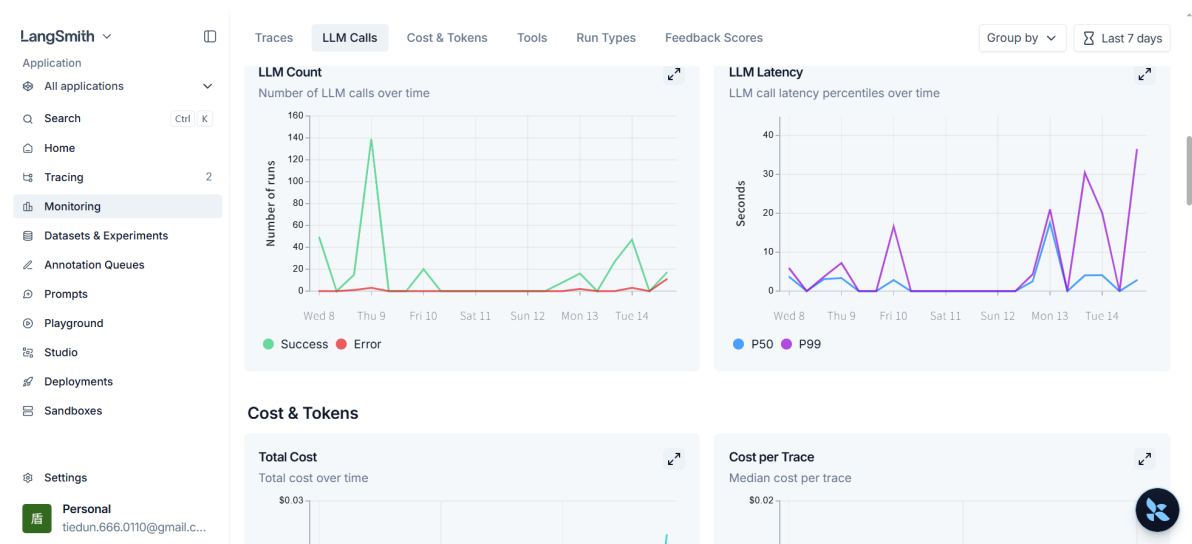
步骤4：查看运行报表



此处提供了大量指标的报表



点击上述标签或下滑页面可以切换指标。



自行探索。